System Design Interview Guide

For Senior Software Engineers — 8+ Years

Stack focus: Java, Spring, AWS, Kafka, Postgres, Oracle, React, Node

Mix of conceptual and scenario-based questions with model answers

How to Use This Guide

Each question follows the same structure:

- **Question —** what to ask.
- **What you're testing —** the competency behind the question.
- **Model answer —** what a strong senior engineer would say. Reference, not a script.
- **Follow-up probes —** to push deeper.
- **Signals —** green flags and red flags to watch for.

Suggested 60-minute flow

- 0–5 min: Background and warm-up
- 5–20 min: Two conceptual questions to calibrate
- 20–55 min: One deep scenario-based design problem
- 55–60 min: Candidate questions

At 8+ years, depth, trade-off reasoning, and operational thinking matter much more than naming the right technology. A candidate who picks 'the wrong' database but explains the trade-offs clearly is stronger than one who picks the 'right' one because they read about it on a blog. Watch for: do they ask about scale, SLAs, and access patterns before they start drawing boxes? Do they think about failure modes and observability, or just the happy path?

Part 1 — Conceptual Questions

Use these to calibrate fundamentals. 5–8 minutes each. Push back on anything that sounds recited.

Q1. Explain the CAP theorem. When have you chosen AP over CP in a real system, or vice versa?

What you're testing

Whether they understand CAP beyond the textbook — that the trade-off only matters during a network partition, and that real systems mix C and A across components.

Model answer

CAP says that during a network partition, a distributed system must choose between consistency (all nodes see the same data) and availability (every request gets a response). Partition tolerance isn't optional in a real distributed system, so CAP is really a partition-time trade-off between C and A.

In practice no production system is purely CP or AP at the whole-system level. You make this choice per data class. On systems I've worked on:

- Payments, inventory decrement, order placement — CP. We'd rather reject a request than double-charge or oversell. Postgres or Oracle with appropriate isolation, or DynamoDB with strongly consistent reads.
- Product catalog, user profile, recommendations, social feed — AP. A few seconds of staleness is fine; downtime isn't. DynamoDB eventually consistent, or Cassandra.

On a checkout flow I built, the cart sat in Redis (AP) but 'reserve inventory' was a Postgres transaction (CP). The hand-off between them was the hard part — we made every step idempotent with a client-supplied idempotency key so retries during partitions were safe.

Follow-up probes

- Difference between eventual consistency and strong consistency from the application's point of view?
- Where does Kafka sit on CAP? (Answer: CP within a partition by default — acks=all, min.insync.replicas — but you can tune toward AP.)
- What is PACELC and what does it add?

Signals

- **Green:** Distinguishes per-data-class trade-offs. Brings up idempotency or PACELC unprompted. Mentions a real system.
- **Red:** 'Pick two of three.' No concrete example. Confuses CAP with latency trade-offs.

Q2. Walk me through how you'd design caching for a read-heavy API. What failure modes would you plan for?

What you're testing

Operational depth. Anyone can say 'add Redis.' A senior should immediately think about invalidation, stampedes, and what happens when the cache itself fails.

Model answer

Three questions first: read/write ratio, tolerable staleness, expected hit rate. Those drive the strategy.

Pattern choice:

- **Cache-aside (lazy load):** app checks cache, falls through to DB on miss, writes back. Simple but the first request after expiry takes the full DB hit.
- **Read-through / write-through:** cache sits in front of the DB. Cleaner but adds a hard dependency on the cache being up.
- **Write-behind:** writes go to cache first, async flush to DB. Fast, but you can lose data on cache failure.

With Spring I'd use Spring Cache abstraction (@Cacheable / @CacheEvict) over a Redis or ElastiCache backend, with method-level cache keys derived from arguments.

Failure modes:

- **Thundering herd / cache stampede:** a hot key expires and hundreds of requests hit the DB. Mitigations: probabilistic early expiration, request coalescing (single-flight per key), or a short lock around the refill.
- **Cache penetration:** lookups for keys that never exist. Cache negative results with short TTL, or use a Bloom filter.
- **Cache avalanche:** many keys expire at once. Add jitter to TTLs.
- **Stale data:** DB updated, cache stale. Either write-through, or explicit invalidation on write, or accept and tune TTL down. For multi-region we sometimes published invalidations over Kafka.
- **Cache outage:** circuit-break to DB-direct with aggressive rate limiting, otherwise we take the DB down too.

Follow-up probes

- How would you size the cache and pick TTLs?
- LRU vs LFU vs TTL-only — when do you pick which?
- Tell me about an incident a cache caused.

Signals

- **Green:** Brings up stampede/avalanche without prompting. Talks about graceful degradation. Mentions invalidation as the hard problem.
- **Red:** Says 'cache invalidation is easy.' Doesn't think about what happens when Redis is down.

Q3. How do you decide between Postgres, Oracle, and a NoSQL store like DynamoDB for a new service? When have you regretted the choice?

What you're testing

Decisions driven by access patterns and operational reality, not hype. The 'regret' framing forces honesty.

Model answer

Three drivers: access pattern, consistency needs, and how the schema is likely to evolve.

My default is Postgres for anything transactional, anything with relationships I'll need to traverse, or anything where the access pattern isn't fully known yet. Joins, transactions, JSON columns when I need flexibility, and great tooling.

Oracle in places I've worked is usually legacy or enterprise-mandated — strong on advanced features (partitioning, materialized views, PL/SQL, RAC) but the licensing cost and operational weight mean I wouldn't choose it for a new greenfield service unless the org already runs it. If I inherit Oracle, I lean into its strengths: partitioning for large tables, AWR for performance analysis, and stored procedures for batch-heavy work.

DynamoDB I reach for when the access pattern is well-understood and one of these is true: key-value lookups at massive scale (sessions, user profiles), single-digit-millisecond latency at any scale, or I want zero ops. The trade-off is that you must design the table around the access pattern up front — no ad-hoc queries.

Regret story: on one project we picked a document store for a transactional workload because 'schema flexibility.' Six months in, we were doing application-level joins, manually maintaining consistency across collections, and reinventing transactions badly. We migrated to Postgres and the code shrank. The lesson: schema-on-read sounds free but you pay for it every query.

Follow-up probes

- 100x scale-up in writes on Postgres — what's your playbook? (Read replicas, partitioning, then sharding or move hot tables out.)
- When does sharding become necessary, and how do you pick a shard key?
- Postgres vs Oracle for a new service — what specifically would tip you to Oracle?

Signals

- **Green:** Talks about access patterns first. Has a real regret story. Doesn't dismiss any option ideologically.
- **Red:** 'NoSQL scales, SQL doesn't' — outdated. No regret story (either inexperienced or not self-reflective).

Q4. Explain Kafka's delivery semantics — at-most-once, at-least-once, exactly-once. How do you achieve each in practice?

What you're testing

Kafka is in their stack. This separates people who consume Kafka tutorials from people who've debugged duplicate events in production.

Model answer

Delivery semantics span the producer, the broker, and the consumer — you have to reason about all three.

- **At-most-once:** producer fires and forgets (acks=0), consumer commits offset before processing. Messages can be lost but never duplicated. Acceptable for high-volume metrics or logs where loss is tolerable.
- **At-least-once (the common default):** producer with acks=all and retries enabled, consumer commits offset after processing. Messages are never lost but can be duplicated — for example,

if a consumer processes a message and crashes before committing the offset, it'll be redelivered. The application must be idempotent.

- **Exactly-once:** Kafka supports this within Kafka — using the idempotent producer (enable.idempotence=true) and transactions that span the consume-process-produce cycle (read_committed isolation, transactional.id on the producer). This works for Kafka-to-Kafka pipelines. Crucially, it does not extend to external systems like Postgres or an external API — those need application-level idempotency.

In practice, even with EOS available, I default to at-least-once + idempotent consumers. It's simpler, the failure modes are easier to reason about, and idempotency is something you want anyway for retries, replays, and disaster recovery. The pattern I use: every message carries a business key or message ID, and the consumer's first action is an upsert keyed on it (e.g., INSERT … ON CONFLICT DO NOTHING in Postgres). Duplicates become no-ops.

Follow-up probes

- How would you handle a poison message that always crashes the consumer?
- Consumer is slower than producer — what knobs do you turn?
- Why doesn't Kafka EOS extend to a downstream Postgres write?

Signals

- **Green:** Notes that EOS is bounded to Kafka. Mentions idempotency as the real solution. Talks about offset commit ordering.
- **Red:** Claims EOS solves everything. Doesn't know what acks=all means. Can't explain why duplicates happen.

Q5. What is idempotency? Why does it matter in distributed systems, and how do you implement it for a POST endpoint?

What you're testing

This question separates middle-level engineers from senior ones. At 8+ years they should treat idempotency as table-stakes.

Model answer

An operation is idempotent if applying it once or applying it many times produces the same result. GETs are naturally idempotent; POSTs that create resources are not.

It matters because every network call between services can fail in the worst possible way — the request succeeded server-side, but the response was lost. The client must retry, and without idempotency, retries create duplicates. This compounds in event-driven systems where the same message can be replayed.

For a POST 'create payment' endpoint, the pattern I use:

- Client generates an idempotency key (UUID) and sends it as a header (Idempotency-Key).
- Server stores the key in a table with a unique constraint, along with the request hash and the eventual response.
- On retry with the same key: if the operation completed, return the stored response. If it's still in progress, return 409 or wait. If the request body differs from what was first seen for that key, reject with 422 — protects against client bugs.

- Keys have a TTL (24–48 hours is typical) so the table doesn't grow forever.

In Spring this lives as a filter or an aspect, with the idempotency store backed by Postgres or Redis. Stripe's API is the well-known reference implementation.

Follow-up probes

- What if two concurrent requests come in with the same idempotency key?
- How does this work across a service mesh with retries baked in?
- How would you make a Kafka consumer idempotent?

Signals

- **Green:** Talks about the request hash check. Knows what to do on concurrent same-key requests (lock the row, or unique constraint). Mentions TTL.
- **Red:** 'Just check if it exists' — misses the response-replay aspect.

Q6. When would you choose synchronous REST between services vs asynchronous messaging via Kafka?

What you're testing

Architectural taste. Both have a place; choosing wrong creates either coupling nightmares or debugging nightmares.

Model answer

REST (or gRPC) when:

- The caller needs the result to continue (validation, lookup, computation).
- The interaction is request/response by nature.
- Low latency matters more than decoupling.
- The set of consumers is small and known.

Kafka when:

- The producer shouldn't care who consumes — fan-out to many subscribers.
- The downstream is slower than upstream and you want to buffer.
- You need an audit log or event-replay capability — Kafka is durable storage too.
- You want to decouple deployment cadences across teams.

The trap I see: teams put Kafka between every two services because 'event-driven is modern,' and end up reinventing synchronous request/response on top of two topics (one for the request, one for the reply, correlation IDs everywhere). That's worse than REST. The right test: does the caller need the answer right now? If yes, REST. If the answer can arrive later or never, Kafka.

A hybrid pattern that works: REST for the synchronous user-facing path, Kafka for the side-effects (audit, notifications, downstream projections, analytics). The user gets a fast response, the bus carries the rest.

Follow-up probes

- If service A calls B which calls C synchronously, what's the failure-mode risk and how do you mitigate?
- How do you handle a Kafka consumer that's been down for 12 hours and is now catching up?

Signals

- **Green:** Calls out the 'reinvented sync over async' anti-pattern. Mentions the hybrid pattern.
- **Red:** Dogmatic — 'always async' or 'always sync.'

Q7. How do you handle database schema migrations in a service that can't have downtime?

What you're testing

Operational maturity. They've shipped enough to know that 'ALTER TABLE' on a 200M-row Postgres table at 2 PM on a Tuesday is a career event.

Model answer

The core rule: every schema change must be backward and forward compatible across the deployment window, because old and new code will run simultaneously.

The standard pattern is expand-and-contract, executed across multiple deploys:

- **Expand:** add the new column, table, or index as nullable / optional. Deploy this alone. Old code ignores it; no break.
- **Migrate:** backfill data, dual-write from the application (write to both old and new columns).
- **Switch reads:** deploy code that reads from the new column.
- **Contract:** stop writing to the old column, then drop it. Often weeks later.

Specifics for Postgres: adding a NOT NULL column with a default rewrites the whole table (mitigated since Postgres 11 for constant defaults). Adding an index without CONCURRENTLY locks writes. Renaming a column is two columns + dual-write, never a direct rename.

Oracle has similar pitfalls — long-running DDL on partitioned tables can be fine if you ONLINE the operation, but be wary of dependent invalidations in PL/SQL packages.

We manage migrations with Flyway or Liquibase, committed alongside code. Each migration is reviewed for: lock impact, runtime estimate on production-sized data, and rollback path. For large backfills, I run them as batched background jobs (10k rows at a time, throttled), never in a single transaction.

Follow-up probes

- How do you roll back a migration that's already been deployed to production?
- Tell me about a migration that went wrong.

Signals

- **Green:** Knows expand-contract. Mentions CONCURRENTLY, batched backfills, dual-write. Has a war story.
- **Red:** 'Take downtime' or 'run it in a maintenance window' as the only answer.

Part 2 — Scenario-Based Questions

These are open-ended. Spend 25–40 minutes on one of them. Don't give the requirements all at once — let the candidate ask. Strong candidates clarify scope before drawing boxes.

QS1. Design a URL Shortener like bit.ly that handles 100M new URLs per month and 10B redirects per month.

What you're testing

Foundational scaling math. Read/write asymmetry. ID generation under load. CDN/cache reasoning. This is the standard warm-up scenario for a reason — easy to start, lots of depth on follow-ups.

What they should ask before designing

- What's the read/write ratio? (~100:1 here.) Tells them this is read-heavy and caching matters.
- How long must short URLs live? Forever? (Affects DB sizing.)
- Do we need custom aliases (vanity URLs)?
- Analytics — click counts, geo, referrer? Real-time or batch?
- What's the SLA on redirect latency? (Usually <100 ms p99.)

Model answer — high-level architecture

Two paths: write path (shorten) and read path (redirect).

Write path:

- API Gateway → Spring Boot service on ECS or EKS behind an ALB.
- ID generation: pre-allocated ID ranges per pod from a central counter table (or Snowflake-style IDs). Encode the integer as base62 → 7 characters covers 3.5 trillion URLs.
- Persist {short_code, long_url, created_at, user_id, expires_at} in Postgres. Use short_code as the primary key.
- Async write to Kafka topic 'url-created' for downstream analytics, search indexing, etc.

Read path:

- Redirects go straight to a lightweight service (could be a Lambda@Edge or CloudFront function for the hot path).
- Layer 1: CloudFront cache — most short URLs are accessed many times, so cache hit ratio should be very high.
- Layer 2: ElastiCache Redis — for misses out of CloudFront.
- Layer 3: Postgres lookup by short_code (primary key, O(log n)).
- Return 301 (permanent) or 302 — 302 is preferred if we want to count every click, since 301 lets the browser cache the redirect forever.
- Asynchronously emit a click event to Kafka for analytics.

Scaling math (back of envelope)

- 100M URLs/month → ~40 writes/sec average, peak maybe 400/sec. Trivial for Postgres.
- 10B redirects/month → ~4,000 reads/sec average, peak maybe 40,000/sec. Far beyond a single Postgres but trivial for CloudFront + Redis.

- Storage: 100M/month × 12 months × 5 years × ~500 bytes ≈ 3 TB. One Postgres instance with read replicas handles this; partition by created_at if it grows.

Failure modes and depth

- **Hot key (viral URL):** CloudFront + Redis absorb it. If Redis becomes a bottleneck, replicate or shard by short_code.
- **ID collisions:** if using hash-based IDs (e.g., MD5 + base62), retry on unique-constraint violation. With pre-allocated ranges, collisions are impossible.
- **Analytics lag:** fine. Click events go to Kafka → batch into S3 → Athena or Redshift. No real-time requirement usually.
- **Abuse / spam:** rate-limit creation per IP/user, scan long URLs against Google Safe Browsing, accept reports.

Follow-up probes

- How do custom aliases work alongside generated IDs? (Reserve a flag column, check the column on insert, same lookup path.)
- Owner of a URL wants real-time click counts — how?
- Now 10x the scale — 100B redirects/month. What changes?
- How do you handle a short URL that needs to expire and 404 afterward?
- Where would you put rate limiting and how?

Signals

- **Green:** Asks for scale first. Separates read and write paths. Picks 7 chars after doing the base62 math. Brings up CDN. Discusses 301 vs 302.
- **Red:** Jumps to 'use a NoSQL DB' without justifying. Forgets caching. Generates IDs with UUID and doesn't realize the lookup path is unchanged but URLs are now 22 chars.

QS2. Design a payment processing system for a marketplace — buyers pay sellers, with a platform fee. It must handle 5,000 transactions per second at peak, never double-charge, and never lose a payment.

What you're testing

Consistency, idempotency, distributed transactions, the saga pattern. This is the question that exposes whether someone has actually built financial systems vs only read about them.

What they should ask before designing

- Sync or async settlement? (Usually async — auth is sync, capture and payout are async.)
- Which payment processors? (Stripe, Adyen, Braintree — we're integrating, not building the cards layer.)
- What's the refund / dispute story?
- Compliance — PCI scope, audit retention?
- Multi-currency?

Model answer — high-level architecture

Services (Spring Boot, all behind an internal API gateway):

- payments-api: receives the buyer's intent, validates, returns a client secret to the SDK.
- payment-orchestrator: drives the saga across processor + ledger + payout.
- ledger: append-only double-entry ledger in Postgres (or Oracle if the org standardizes there).
- payout-service: schedules and executes seller payouts.
- processor-adapter: wraps the external payment processor's API.

Storage:

- Postgres for the ledger, with strict isolation. Every row is immutable; corrections are compensating entries. Partitioned by month.
- Kafka for inter-service events — payment.authorized, payment.captured, payment.failed, payout.queued, payout.completed.
- Redis for idempotency-key storage with TTL.

The critical patterns

Idempotency: Every external call (to the processor) and every internal API accepts an idempotency key. Stored in Postgres with a unique constraint. Retries return the original response. The processor's own idempotency keys are forwarded so retries don't double-charge.

Saga, not 2PC: A payment crosses our system, the processor, and the ledger. 2PC across these is impossible (we don't control the processor). Instead, an orchestrator runs a saga: authorize on processor → write 'authorized' to ledger → capture on processor → write 'captured' to ledger. Each step has a compensating action (void, refund, reverse ledger entry). State machine persisted in Postgres.

Outbox pattern: When the ledger commits a row, in the same Postgres transaction we write to an 'outbox' table. A separate process tails the outbox and publishes to Kafka. This guarantees the DB and Kafka stay in sync without 2PC. Debezium with CDC on the outbox table is the clean implementation.

Exactly-once accounting: Every ledger row has a globally unique business key (payment_id + step). Inserts use ON CONFLICT DO NOTHING. A retry of the same logical operation is a no-op.

Failure modes

- **Processor returns timeout:** we don't know if charge succeeded. Saga retries with same idempotency key. If still timing out, reconciliation job runs every N minutes querying the processor for unknown-status payments.
- **Our service crashes mid-saga:** saga state in Postgres is the source of truth. On startup, a recovery process resumes incomplete sagas.
- **Kafka outage:** writes still succeed (outbox absorbs). Downstream consumers catch up after recovery.
- **Dispute / chargeback weeks later:** ledger is append-only, so we record a compensating entry; original entry is preserved for audit.

Scaling considerations

- 5,000 TPS is moderate. Postgres on RDS r6i.4xlarge handles this with proper indexes and partitioning.
- The bottleneck is usually the processor's rate limits — we batch where possible, queue when throttled.
- Hot accounts (a viral seller): we shard the ledger by account_id hash if needed, but most marketplaces don't reach this.

Follow-up probes

- Walk me through what happens when authorize succeeds but capture fails.
- Why the outbox pattern and not just write to Kafka after the DB commit?
- How do you reconcile your ledger against the processor's records?
- How would you support refunds and partial refunds?
- Suppose the orchestrator processed the same saga twice in parallel — what stops a double-charge?

Signals

- **Green:** Reaches for outbox pattern unprompted. Designs the ledger as append-only. Talks about reconciliation. Knows why 2PC won't work here.
- **Red:** Says 'use distributed transactions.' No reconciliation job. Treats the processor as infallible. Mutable ledger rows.

QS3. Design a notification system that sends emails, SMS, and push notifications. Volume: 50M notifications/day, with bursts of 1M/minute during marketing campaigns. Some notifications are transactional (password reset — must be fast and reliable) and some are marketing (must respect user preferences and quiet hours).

What you're testing

Queue design, prioritization, fan-out, rate limiting against third-party APIs, dealing with bursty traffic.

What they should ask

- Latency SLA for transactional? (Seconds.) For marketing? (Minutes to hours is fine.)
- Which providers? (SES, SNS, Twilio, FCM.)
- Do we track delivery, opens, clicks?
- User preferences — opt-out, quiet hours, channel preference?

Model answer — high-level architecture

Two ingest paths into a unified pipeline:

- Transactional API: synchronous, validates, writes to high-priority Kafka topic 'notifications.transactional'.
- Marketing API / batch ingest: writes to 'notifications.marketing' topic, often via a scheduled job from a campaign tool.

Core pipeline (Spring Boot consumers on ECS):

- preferences-resolver: enriches each event with the user's channel preferences, opt-out status, locale, timezone. Filters out events that violate user choices or quiet hours (re-queues them with a delay).
- templating-service: renders the message body. Templates versioned in S3 + cached locally.
- dispatcher: per-channel — email-dispatcher, sms-dispatcher, push-dispatcher. Each handles provider-specific rate limits and retries.

Storage:

- Postgres for user preferences and template metadata.
- DynamoDB for per-notification status (delivered, opened, bounced) — high write volume, simple key lookup.
- Redis for rate-limit counters per provider.

Prioritization and bursting

Separate topics is the cleanest way to prioritize. Transactional consumers always have headroom; marketing consumers scale up during campaigns but are starved first if backed up.

For the 1M/minute burst: consumer groups auto-scale (KEDA on Kubernetes, or ECS scaling on Kafka lag). The bottleneck won't be us — it'll be the provider's rate limit. SES allows ~14 emails/sec per account by default, so we'd raise limits in advance, distribute across SES regions, or fall back to a secondary provider for overflow.

Reliability patterns

- **Retries with backoff:** exponential, capped. Permanent failures (invalid email, blocked number) go to a DLQ for review.
- **Provider failover:** if SES has elevated bounce rate or 5xx, route to a secondary (SendGrid). Circuit breaker on each provider adapter.
- **Webhooks back from providers:** delivery, bounce, complaint events come back as webhooks → ingest to Kafka → update status in DynamoDB. Bounces auto-update user preferences (hard bounce = mark address invalid).
- **Deduplication:** every notification has a unique ID; dispatchers use it as the provider-side idempotency key.

Quiet hours and preferences

Quiet hours are user-timezone-relative. When the preferences-resolver detects a quiet-hour conflict for a marketing message, it re-queues with a delay (Kafka has no native delay queue; we use a 'scheduled' topic where a consumer polls only ready messages, or use SQS delay queues for this part).

Follow-up probes

- How do you make sure a password reset email isn't delayed by a marketing campaign in the queue?
- Walk me through your DLQ strategy — when do humans look at it?
- How do you respect GDPR — user requests deletion of all their notification history?
- How would you A/B test marketing template variants?
- What happens when DynamoDB throttles?

Signals

- **Green:** Separate topics for prioritization. Provider rate limit as the real bottleneck. Webhook-driven status updates. DLQ strategy.
- **Red:** Single queue for everything. No provider failover. Doesn't mention rate limits. Builds quiet-hour logic as a cron job that scans the DB.

QS4. Your team owns a Spring Boot monolith on AWS, serving a React frontend. It's slow, the team is growing, and deploys are scary. Walk me through how you'd evolve it — including what you wouldn't change.

What you're testing

Architectural maturity in the brownfield. Senior engineers must resist the urge to rewrite. The right answer is incremental, hypothesis-driven, and often boring.

What they should ask

- Where exactly is it slow — request latency, build time, deploy time, time-to-onboard?
- What does the team structure look like? How many engineers, how many teams?
- What's the current deploy frequency and rollback story?
- What's the test coverage and what's the CI duration?
- What's the business roadmap? Are we adding features fast or stabilizing?

Model answer — frame the problem first

I'd start by separating three distinct things people lump under 'the monolith is slow':

- Runtime slowness — actual user-facing latency.
- Development slowness — long CI, painful local setup, merge conflicts.
- Deployment fear — every release is risky because the blast radius is the whole product.

Each has a different remedy. Conflating them leads to 'let's microservice everything,' which solves none of them and adds distributed-systems problems on top.

My playbook, in order

1. Observability first. Before changing anything: distributed tracing (OpenTelemetry → Jaeger or X-Ray), structured logs to CloudWatch, RED metrics (rate/errors/duration) per endpoint in Prometheus + Grafana. Without this, every later decision is a guess. This usually takes 2–4 weeks and pays for itself immediately by finding the worst N+1 queries.

2. Fix the runtime issues in-place. Most 'slow monoliths' have a handful of culprits: N+1 queries, missing indexes, synchronous calls that should be async, oversized JSON payloads, JVM tuning. Profile, fix, measure. This often gets a 5–10x improvement without architectural change.

3. Strengthen the deployment pipeline. Blue/green or canary on ECS or EKS, feature flags (LaunchDarkly or self-hosted with Unleash), automated rollback on error-rate spike. Deploys become non-scary even before the monolith shrinks.

4. Modularize within the monolith. Spring Modulith, or just ruthless package boundaries with ArchUnit rules. Identify domain boundaries (DDD bounded contexts). The team feels the benefits — fewer merge conflicts, clearer ownership — without taking on network latency or distributed transactions.

5. Extract only when a module hurts. A module deserves extraction when it has a distinct scaling profile, a separate team owning it, or a different deployment cadence. Strangler fig: route traffic at the edge to the new service for specific endpoints, keep the rest in the monolith. Don't extract for ideological reasons.

6. Frontend split if needed. If the React app itself is heavy, code-split, lazy-load routes, move heavy pages to their own bundles. Module federation only if multiple teams genuinely need independent deploys of frontend areas.

What I would NOT change

- Postgres / Oracle. The DB is almost never the architectural problem at this stage. Tune queries first.
- The language or framework. Rewriting Spring Boot into Go (or whatever) buys nothing relative to the cost.
- The team structure prematurely. Conway's law works both ways — splitting teams to match a future architecture creates friction without benefits.

Follow-up probes

- When does a monolith genuinely need to be split? What's your tipping point?
- How do you handle the database during extraction — shared DB anti-pattern vs separate DBs with sync issues?
- What's the smallest possible first microservice extraction you'd recommend?
- Sell this incremental plan to a CTO who wants 'a microservices roadmap by Q3.'

Signals

- **Green:** Resists rewrite. Names observability as step zero. Talks about strangler fig and bounded contexts. Has a clear 'what I wouldn't change' list.
- **Red:** Immediately proposes microservices for every domain. Doesn't mention metrics or tracing. Treats the monolith as inherently bad.

QS5. Design a real-time order tracking system. When a buyer places an order, the seller's dashboard updates within 2 seconds, and the buyer's React app shows live status changes (paid → preparing → shipped → delivered). 1M concurrent dashboards open at peak.

What you're testing

Real-time delivery to clients (WebSocket / SSE), fan-out at scale, connection management, integration with an event-driven backend.

Model answer

The frontend uses WebSockets (or Server-Sent Events for one-way updates — simpler than WebSockets, works fine here since clients aren't pushing back). I'd lean SSE for the buyer-side (mostly read) and WebSockets only where I need duplex (e.g., seller responding to chat).

Connection layer: a fleet of Node.js gateways on ECS Fargate behind an NLB (or ALB with WebSocket support). Node holds tens of thousands of long-lived connections per instance cheaply. Each gateway registers its open connections in Redis with a TTL heartbeat (connection_id → user_id → gateway_instance_id).

Backend flow on a status change:

- Order service writes 'order.status.changed' to Kafka.
- A fan-out service consumes the event. For each interested user (buyer + seller), it looks up active connections in Redis.
- It publishes to a Redis Pub/Sub channel keyed by gateway instance, or directly RPCs the gateway.
- The gateway pushes the event over the open socket.

Why not have every gateway consume from Kafka directly? At 1M connections across hundreds of gateway instances, every event would be processed by every gateway. The fan-out indirection means only the gateway(s) holding the relevant connection do work.

Scaling math

- 1M concurrent connections. Node handles ~30–50k per instance comfortably with WebSockets, more with SSE. So 25–50 gateway instances.
- Status changes per second — depends on order volume. At 10k orders/sec each transitioning through ~5 states, that's 50k events/sec — well within Kafka's capacity.
- Fan-out factor is low (each order has 1 buyer, 1 seller) so push volume is also ~50k/sec.

Reliability and recovery

- **Missed messages while disconnected:** client reconnects with last-seen event ID. Server replays from a per-user event log (Kafka with a compacted topic, or a 'recent events' table in DynamoDB with TTL). SSE has a built-in Last-Event-ID header — convenient.
- **Gateway crash:** client reconnects (load balancer routes to a healthy instance), re-registers in Redis, resumes from last event ID.
- **Stale connection entries in Redis:** TTL with heartbeat; if gateway dies, entries expire.
- **Backpressure:** if a slow client can't drain the socket, gateway drops events for that connection past a buffer threshold; client reconciles on reconnect.

Why not just poll?

At 1M dashboards polling every 2 seconds, that's 500k req/sec hitting an API — far more load than push. Push is cheaper at this scale, even with the connection-management complexity.

Follow-up probes

- Why SSE over WebSocket here, or vice versa?
- How does authentication work on a long-lived connection? What about when the JWT expires mid-connection?
- Buyer has the page open on phone and laptop — both should update. How?
- How do you handle a deployment of the gateway tier without dropping every connection?
- Suppose the seller dashboard needs offline support — what changes?

Signals

- **Green:** Picks SSE vs WebSocket deliberately. Designs the gateway-to-fanout indirection. Handles reconnect with Last-Event-ID. Calculates connections-per-instance.
- **Red:** Says 'just use WebSockets' without scoping. Has every gateway consume Kafka directly. No reconnect / catch-up story. Suggests polling.

QS6. An internal Spring Boot service is suddenly timing out under load. Latency p99 has climbed from 200 ms to 8 seconds over two days. CPU is at 40%, memory is fine, the database looks healthy. Walk me through your diagnosis.

What you're testing

Real-world debugging. Not a clean greenfield design — the messy reality of production. Senior engineers should have a mental playbook.

Model answer — diagnostic flow

I work outward from the service, ruling out causes systematically.

1. Confirm the signal. Is this all endpoints or one? All instances or a subset? All users or a tenant? p99 jumped while p50 stayed flat suggests tail-latency / contention, not a uniform slowdown. Two-day climb suggests a leak or accumulating state, not a deployment.

2. Check the obvious. Recent deploys (rolled out 2 days ago?), config changes, feature flag toggles, traffic shape change, downstream service health, dependency version bumps. CloudWatch + git log is the first 10 minutes.

3. JVM diagnostics. 40% CPU and fine memory doesn't mean the JVM is healthy. I'd check: GC behavior (G1 pause times, frequency, old-gen growth — a slow leak shows here), thread count (thread pool exhaustion looks like timeouts but low CPU), and look at a thread dump. If most threads are BLOCKED on a single monitor or WAITING on a DB connection, that's it.

4. Connection pools. HikariCP metrics: active vs idle vs pending. If pending > 0 sustained, the pool is exhausted. Common cause: a downstream call slowed down (DB, third party, internal service), threads hold connections longer, pool empties, queue grows. The service looks idle CPU-wise but is bottlenecked on connections. Same logic applies to HTTP client pools.

5. Downstream dependencies. 'The database looks healthy' — does it? Check from the service's perspective: query latency from this service's traces, not from RDS Performance Insights alone. A specific slow query that's only run from this code path won't show in DB top-N. Also: any third-party API calls? Their slowdown is invisible unless we trace it.

6. Distributed tracing. If we have OpenTelemetry / X-Ray traces, look at a slow request end-to-end. Where's the time going? Usually one span dominates.

7. Resource leak hypothesis. Two-day climb is suspicious. Heap leak (objects accumulating, GC pressure rising) shows in memory and GC. But a non-heap leak — file descriptors, threads, JDBC connections not returned to pool, ThreadLocals accumulating — would match 'memory fine but slowness growing.' Check thread count over time, FD count, connection pool size over time.

Most likely culprits at 8+ years of experience

- Connection pool exhaustion from a downstream slowdown — single most common root cause of this symptom shape.
- A query that's gotten slower because table grew past the point where its index plan stops working (Postgres planner flips to seq scan).
- Thread-local accumulation (often from a recent library upgrade).
- A new noisy neighbor on a shared resource (RDS, cache, network).

Follow-up probes

- Walk me through reading a thread dump.
- How would you tell a connection-pool issue from a real DB slowdown?
- Now suppose it's only happening on one out of four instances. What changes in your approach?
- You've found the bad query. It's been there for two years. What changed?

Signals

- **Green:** Asks clarifying questions before diagnosing. Works outward methodically. Names connection pools, thread dumps, tracing. Has a feel for what symptom shape implies.
- **Red:** Jumps to 'scale it up.' Doesn't ask if a deploy happened. Doesn't know how to read a thread dump.

QS7. You need to migrate a table with 10 million records from one database to another (e.g., Oracle to Postgres, or an old schema to a new one). The source system stays live during migration. How do you guarantee every record is migrated, how do you reconcile the two tables, and how do you find any missing or mismatched records?

What you're testing

This is the single best question for testing operational maturity around data. It blends ETL design, idempotency, dealing with a moving target (writes happen during migration), reconciliation, and the discipline to prove correctness rather than assume it. At 8+ years they should have lived through this.

What they should ask before designing

- Is the source live during migration, or can we take a read-only window? (Live changes everything — they should ask this first.)
- What's the cutover plan — big-bang switchover, or dual-write period with gradual rollover?
- Are the schemas identical, or is there transformation (column renames, type changes, splits, merges)?
- What's the acceptable downtime — zero, minutes, hours?
- Are there foreign-key relationships to other tables that also need to migrate?
- How fresh must the target be at cutover — last-write must be visible, or eventual consistency okay for a window?
- 10M records — what's the row size, total data size? (Determines batch sizes and transfer windows.)

Model answer — overall approach

I treat this as three phases, each with explicit success criteria: bulk backfill, ongoing sync (CDC), and reconciliation. Plus a fourth phase, cutover, that I plan before I write any code.

Phase 1 — Bulk backfill

Goal: copy the current state of the source table to the target.

- Read the source in batches (e.g., 10k rows at a time), ordered by primary key. Never SELECT * with no LIMIT on a 10M-row table — locks, memory blow-up, or just slow.
- For each batch: transform rows if needed, write to target with upsert semantics (INSERT … ON CONFLICT DO UPDATE for Postgres, MERGE for Oracle). Upsert means re-running the batch is safe.
- Pagination by primary key, not OFFSET. OFFSET 9,000,000 is a disaster — the DB still walks 9M rows. Instead: WHERE id > :last_seen_id ORDER BY id LIMIT 10000.
- Checkpoint the last successfully processed PK after each batch in a control table. If the job crashes, resume from there — no double work, no missed rows.
- Throttle. 10M rows at full speed will spike source CPU and replication lag. Add a sleep between batches, or use a token bucket to cap row/sec.
- Run in parallel by PK ranges if needed. Split the keyspace into N shards, one worker per shard. Each shard has its own checkpoint.

- Capture the start timestamp (or LSN/SCN) before backfill begins. Anything written after this needs to be picked up by the CDC phase.

Phase 2 — Ongoing sync (CDC)

The source is live, so rows are being inserted/updated/deleted while you backfill. Without this phase, the target drifts the moment you start.

- Set up Change Data Capture from the start of backfill — Debezium against Postgres logical replication, or Oracle LogMiner / GoldenGate. Stream changes to Kafka.
- CDC events arrive on a Kafka topic, keyed by primary key (so all changes to one row land in the same partition and stay ordered).
- A consumer applies these events to the target with the same upsert pattern. Because every change is idempotent (insert as upsert, update as upsert, delete by PK), replays are safe.
- Crucial ordering rule: the CDC consumer must not overwrite a backfill row with an older event. Solution — every row carries a version (timestamp, LSN, or row version). The target applies updates only if the incoming version is newer (WHERE target.version < incoming.version). This handles the race between backfill and CDC.
- Once backfill completes, CDC catches up to current and then runs in steady state. Lag should be seconds.

Phase 3 — Reconciliation (the core of the question)

Backfill said it copied 10M rows. CDC says it's caught up. But how do you actually prove the target matches the source? Three levels of reconciliation, cheap to expensive:

Level 1 — Count checks

- SELECT COUNT(*) on source and target, ideally as of a known point in time (snapshot). Fast, catches catastrophic problems.
- Bucketed counts: GROUP BY date_trunc('day', created_at) on both sides and diff. Tells you which time range is off.
- Limitation: counts can match while individual rows differ. Necessary but not sufficient.

Level 2 — Checksum / hash comparison

- Compute a hash per row (MD5 or SHA-1 of the concatenated columns) on both sides. Then compare aggregate hashes per PK range.
- Bucket rows into ranges (e.g., 1000 buckets by id % 1000 or by PK range). For each bucket: compute SUM(hashtext(...)) or similar on source and target. Compare bucket-by-bucket.
- Mismatched buckets → drill into individual rows in that bucket only. Cuts a 10M-row reconciliation into manageable pieces.
- Tools that do this well: pt-table-checksum (MySQL world), or roll your own with a SQL query that produces (bucket_id, count, hash) pairs from both sides and joins.

Level 3 — Row-by-row diff for mismatched buckets

- For buckets that disagree, fetch all rows in that bucket from both sides and compare in application code.
- Three outcomes per PK: only in source (missing in target — re-sync), only in target (extra — investigate, possibly a delete that didn't replicate), both but different (re-sync the source value).

- Log every discrepancy with PK, side, and the column that differs. This is your audit trail.

Finding missing records specifically

The classic approach — and what they should describe:

- Snapshot both sides at a known point. If using CDC, pause the consumer briefly so the target is stable, or reconcile against a target snapshot from a moment before CDC's current position.
- Pull primary keys from source into a temp table or file.
- Anti-join: SELECT source.id FROM source LEFT JOIN target ON source.id = target.id WHERE target.id IS NULL → rows missing from target.
- Reverse anti-join for orphans on the target side.
- If both sides are huge, do it in batches by PK range, or export PK lists to S3 and diff in Spark / Athena.

In practice, hash-bucket comparison covers both 'missing' and 'mismatched' in one pass — missing rows show up as bucket count mismatches.

Reconciliation as a continuous job, not a one-shot

During dual-running, run reconciliation continuously (hourly bucket-level checks, daily full-table checksums). Drift is a signal — either CDC missed something, or there's a write path bypassing CDC. Catching it early matters.

Phase 4 — Cutover

- Verify reconciliation passes for several consecutive runs.
- If possible, dual-write from the application for a period — every write goes to both source and target. Reconciliation catches discrepancies in real-time.
- Switch reads to target gradually behind a feature flag (1% → 10% → 100%) while monitoring.
- Switch writes once reads are stable. Keep CDC running in reverse (target → source) as a safety net during the cutover window in case you need to roll back.
- Decommission source only after the rollback window passes — typically a week or more.

Failure modes they should call out

- **Backfill crashed at row 6M:** checkpoint-based resume handles it. No reset needed.
- **CDC consumer lag spike during a write storm:** alert and let it catch up. Target is allowed to be behind during dual-running.
- **Row updated on source during backfill:** version column on the target rejects the older backfilled value when CDC delivers the newer one. Or — if there's no version, do the backfill, then re-run CDC from the start LSN to overwrite anything stale.
- **Hard deletes on source:** if the source uses hard deletes, CDC delivers a tombstone. If for any reason CDC missed it, the target keeps a phantom row. Periodic anti-join from target → source catches these.
- **Schema drift mid-migration:** freeze schema changes during cutover window. If unavoidable, version the transformation logic.
- **Foreign-key dependencies:** migrate parent tables first, defer FK constraints on the target until all data is loaded, or migrate as a coordinated set.

Follow-up probes

- How would you parallelize the backfill safely?
- Suppose the source table has no monotonic primary key — say, UUIDs. What changes?
- CDC consumer fell over for 4 hours. How do you know if you missed anything, and how do you catch up?
- The two tables have different schemas — one column on source is split into three on target. How does reconciliation handle that?
- How do you do this when the source is Oracle and the target is Postgres? What about type differences (NUMBER, DATE, CLOB)?
- How would you test the migration before running it in production?
- If reconciliation finds 200 mismatches out of 10M, what's your decision tree?

Signals

- **Green:** Asks about live-vs-frozen source first. Names CDC unprompted. Knows pagination by PK, not OFFSET. Designs reconciliation in tiers (count → hash → row). Has a version-column story for the backfill/CDC race. Treats reconciliation as continuous, not a one-shot.
- **Yellow:** Knows the bulk-copy part but waves hands at the 'live source' problem. Suggests reconciliation only as 'compare counts' or 'compare a sample.'
- **Red:** Plans to take downtime, copy, switch — without engaging with the question. No reconciliation strategy beyond 'check the count.' Doesn't mention idempotency or resume-from-checkpoint. Doesn't think about ordering between backfill and CDC.

Closing — Evaluating the Candidate

What 'senior' should sound like

- Asks questions before drawing. Establishes scope, scale, SLAs, constraints.
- Talks in trade-offs. 'We could do X or Y. X gives us A, costs us B. Given the requirement, I'd lean X.'
- Mentions failure modes unprompted. Caches fail. Networks partition. Disks fill. Deploys roll back.
- Brings operational concerns into design — monitoring, deployment, rollback, on-call burden.
- Has war stories. Concrete details. Mistakes they made and lessons learned.
- Knows the boundaries of their knowledge. Says 'I don't know' or 'I'd have to check' without flinching.

What's a no-hire signal at this level

- Names technologies without trade-offs. 'Just use Kafka.' 'Just use Kubernetes.'
- Hand-waves at scale numbers. Doesn't back-of-envelope.
- All happy path. No mention of what breaks.
- Can't draw a clear boundary between services or layers.
- No production scars. Eight years and never had an incident is either lucky or not honest.
- Defensive when challenged. A senior should welcome push-back as a way to refine, not defend their first answer.

A simple scoring rubric (1–5)

- **Scope & requirements:** did they clarify before designing?
- **Core design:** is the high-level architecture coherent and appropriate to scale?
- **Trade-offs:** did they discuss alternatives and justify choices?
- **Failure handling:** did they think about what breaks and how to recover?
- **Operability:** monitoring, deployment, debugging, on-call?
- **Communication:** clarity, structure, response to push-back?

Average ≥ 4 across these is a strong hire. Average 3 with one or two 4s is likely a hire depending on team. Anything with a 2 in 'trade-offs' or 'failure handling' is a concern at the 8+ year level.